



Rechnernetze, Übungsblatt 1, Sommer 2025

Abgabe

Die Abgabe in Form eines Pull-Requests in das Repository <https://github.com/syssoft-ds/TokenRingUDP-2025S> muss bis zum 30.04.2024 um 23:59 Uhr geschehen.

Aufgabe 1: Token Ring

Klonen Sie das Repository „TokenRingUDP-2025S“ (<https://github.com/syssoft-ds/TokenRingUDP-2025S>). Importieren Sie anschließend das enthaltene Java-Projekt in eine IDE Ihrer Wahl (eine hoffentlich auch für Sie funktionstüchtige IntelliJ-Konfiguration ist Teil des Repositories). Versuchen Sie den Source-Code zu verstehen und nutzen Sie bei Fragen und Problemen die Chat-Komponente des „Github Copilot“ oder eine andere generative KI. Versuchen Sie das Programm zu übersetzen und zu starten. Bei Problemen können Sie für erste Experimente auch direkt das fertige Fatjar im Wurzelverzeichnis des Projekts ausführen. Die Verwendung wird in der Datei *README.md* beschrieben.

Aufgabe 2: Teamwork

Wagen Sie sich zusammen mit anderen Vorlesungsteilnehmern an einen möglichst großen Ring. Die Protokollausgaben der Programme dokumentieren die erreichte Größe und die IP-Adressen der beteiligten Rechner.

Aufgabe 3: WireShark

Installieren Sie die aktuelle Version des „Paket Sniffers“ WireShark. Fangen Sie mit der Hilfe dieses Programms einzelne Nachrichten eines Token-Rings aus Aufgabe 1 (lokal nur auf einem Rechner) sowie Aufgabe 2 (verschiedene Teilnehmer) ein. Machen Sie sich mit der Filter-Syntax von Wireshark vertraut, um nur die Token-Ring-Nachrichten darzustellen (Display Filter) oder überhaupt nur diese einzufangen (Capture Filter).

Aufgabe 4: Git(Hub)

Protokollieren Sie in Form eines kurzen Berichts Ihre Ergebnisse zu den Aufgaben 2 und 3 als Teil des Repositories. Beschreiben Sie die Probleme, auf die Sie gestoßen sind, und wie Sie diese behoben haben, bzw. welche Schritte Sie unternommen haben.

Geben Sie dieses Ergebnis als Pull-Request über GitHub zurück. Wir werden Ihre Ergebnisse in Form einzelner Branches in das Repository integrieren. Geben Sie deshalb im Kommentar zum Pull-Request den von Ihnen bevorzugten Namen Ihres Branches an. Achten Sie darauf, dass keine persönlichen oder sensiblen Informationen in Ihr Repository gelangen, da diese durch den Pull-Request ggf. für alle sichtbar werden!

Sollten Sie einen Fehler in der Token-Ring-Implementierung finden oder einen sinnvollen Verbesserungsvorschlag haben, implementieren Sie diesen Vorschlag oder Bugfix gerne und leiten uns diesen als Teil Ihres Pull-Requests weiter. Trauen Sie sich ruhig, mit den Möglichkeiten von Git und GitHub zu experimentieren!

zu Aufgabe 2

Ring via local network

IP Adressen: 192.168.1.7
192.168.1.5

Grösse: 2

[Protokoll ausgabe]

Sending {"sequence": 1538, "ring": {"ip": "192.168.1.7", "port": 52809}, {"ip": "192.168.1.5", "port": 60431}} to 192.168.1.5:60431
Received "sequence":1539, "ring":{"ip": "192.168.1.5", "port": 60431, {"ip": "192.168.1.7", "port": 52809}} from 192.168.1.5:60431
Token: seq=1539, members=2 (192.168.1.5, 60431)(192.168.1.7, 52809)

Ring via VPN

IP Adressen: 136.199.4.31
136.199.4.47

Grösse: 2

[Protokoll ausgabe]

Sending {"sequence":137,"ring": {"ip": "136.199.4.31", "port": 54053}, {"ip": "136.199.4.47", "port": 57561}} to 136.199.4.47:57561
Received {"sequence":138,"ring": {"ip": "136.199.4.47", "port": 57561}, {"ip": "136.199.4.31", "port": 54053}} from 136.199.4.47:57561
Token: seq=138, #members=2 (136.199.4.47, 57561)(136.199.4.31, 54053)

Ring via Internet

IP Adressen: 192.168.1.7 (lokal)
192.168.178.40 (87.167.179.106)

Grösse: 2

[Protokoll ausgabe]

Received {"sequence":0,"ring":{"ip" : "192.168.178.40", "port" : 64575}} from
87.167.179.106:64575

Token: seq=0, members=1 (192.168.178.40, 64575)

Sending {"sequence":1,"ring":{"ip" : "192.168.178.40", "port" : 64575}, {"ip" :
"192.168.1.7", "port" : 44541}}} to 192.168.1.7:44541

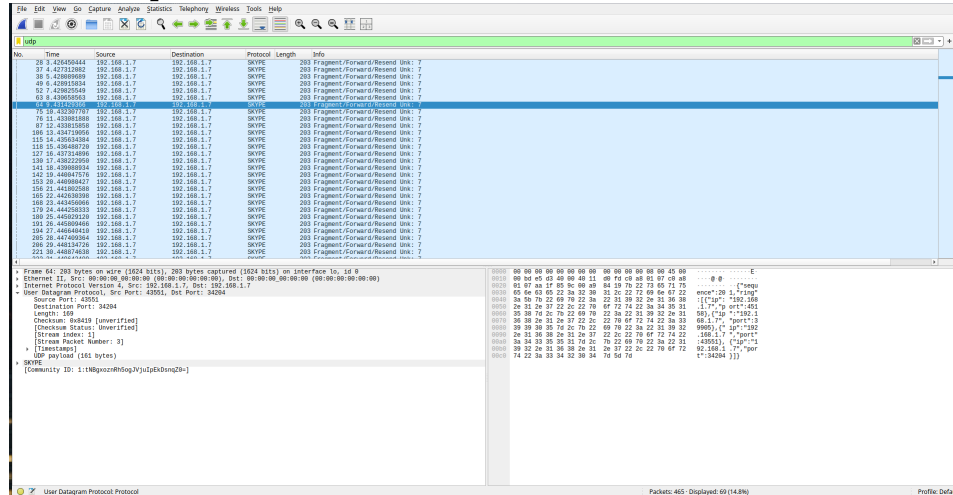
Received "sequence":1,"ring":{"ip" : "192.168.178.40", "port" : 64575}, {"ip" :
"192.168.1.7", "port" : 44541}}} from 192.168.1.7:44541

Token: seq=1, members=2 (192.168.178.40, 64575) (192.168.1.7, 44541)

zu Aufgabe 3

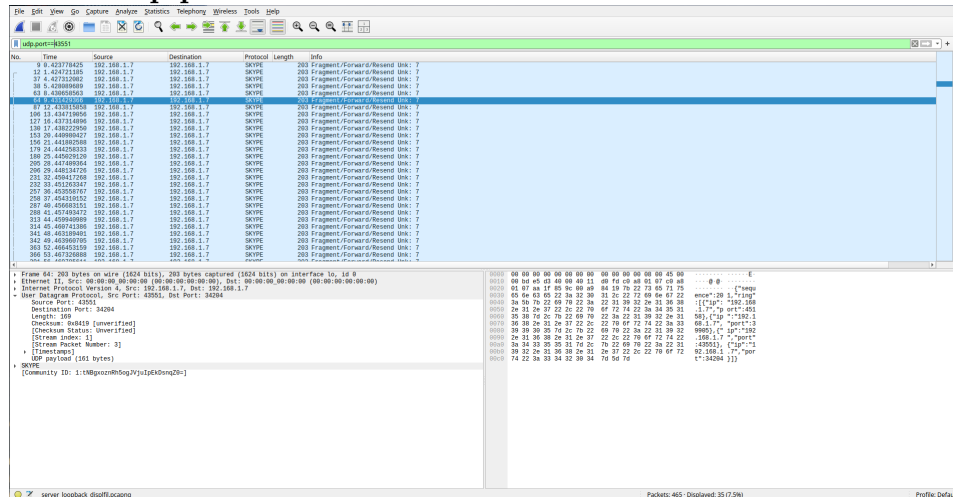
Ring loopback
display filter

filter: udp



Dieser Anzeigefilter zeigt nur udp Pakete an.

filter: udp.port==43551



Dieser Anzeigefilter zeigt nur udp Pakete an, die port 43551 verwenden.

filter: udp.port==34204

Wireshark packet capture showing a list of UDP packets filtered by port 34204. The packet list shows various source and destination IP addresses and ports. The packet details pane shows the structure of a UDP packet, including Ethernet II, Internet Protocol Version 4, and User Datagram Protocol. The packet bytes pane shows the raw data in hexadecimal and ASCII.

Dieser Anzeigefilter zeigt nur udp Pakete an, die port 34204 verwenden.

filter: ip.src==192.168.1.7 and udp.length==169 and udp.checksum==0x8419

Wireshark packet capture showing a list of UDP packets filtered by source IP 192.168.1.7, length 169, and checksum 0x8419. The packet list shows various destination IP addresses and ports. The packet details pane shows the structure of a UDP packet, including Ethernet II, Internet Protocol Version 4, and User Datagram Protocol. The packet bytes pane shows the raw data in hexadecimal and ASCII.

Dieser Anzeigefilter zeigt nur die udp Pakete an, die von der IP Adresse: 192.168.1.7 gesendet werden, eine Länge von 169 byte besitzen und deren checksumme 0x8419 ist.

capture filter

filter: udp

Wireshark interface showing a packet capture filter 'udp' applied. The packet list shows 62 packets, all of which are UDP fragments. The packet details pane shows the selected packet (No. 1) as a UDP fragment from 192.168.1.7 to 192.168.1.7, port 43552 to 34204. The packet bytes pane shows the raw data of the packet.

Dieser Filter lässt nur udp Pakete durch.

filter: udp and src 192.168.1.7 and not dst port 38161

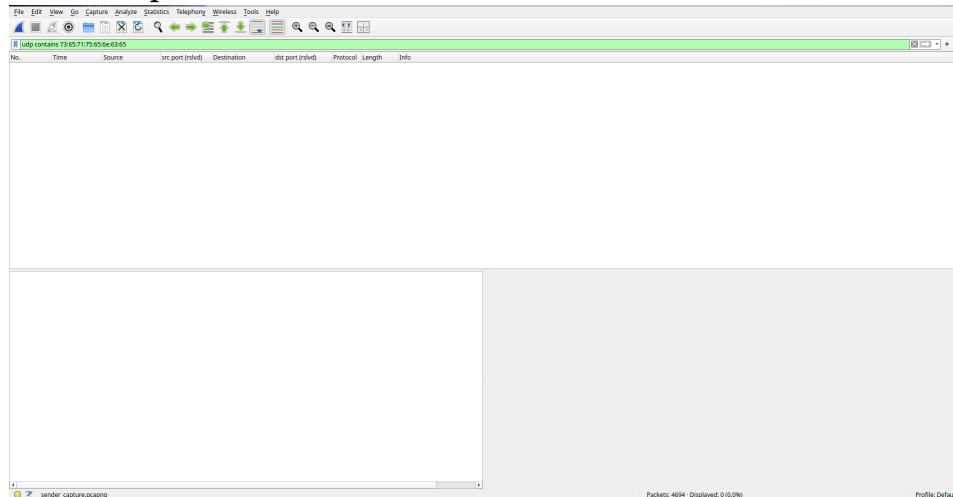
Wireshark interface showing a packet capture filter 'udp and src 192.168.1.7 and not dst port 38161' applied. The packet list shows 73 packets, all of which are UDP fragments. The packet details pane shows the selected packet (No. 1) as a UDP fragment from 192.168.1.7 to 192.168.1.7, port 43552 to 34204. The packet bytes pane shows the raw data of the packet.

Dieser Filter lässt nur udp Pakete durch, die die Ursprungsadresse 192.168.1.7 haben und nicht an port 38161 gesendet werden.

Ring: VPN

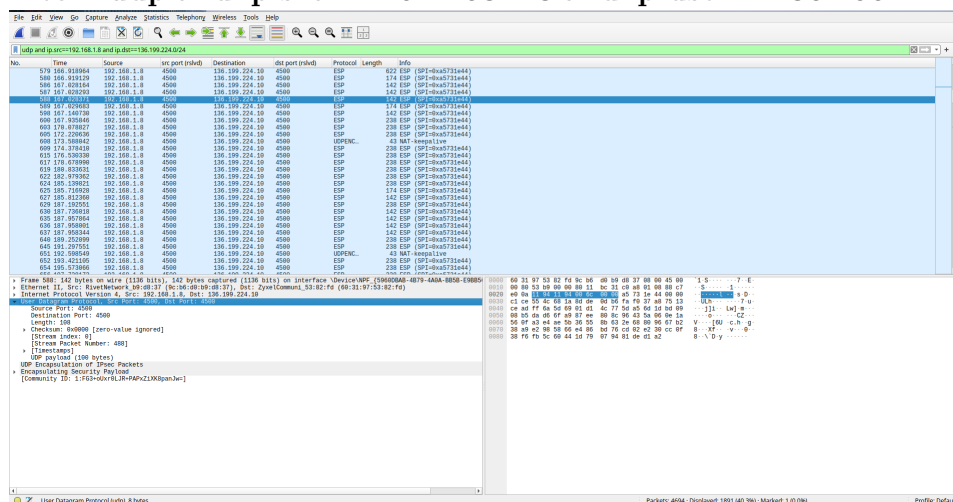
display filter

filter: udp contains 73:65:71:75:65:6e:63:65



Dieser Filter lässt nur udp Pakete durch, die die Bytesequenz 73:65:71:75:65:6e:63:65 enthalten. Das ist die Bytefolge für das Wort *Sequence*. Der Filter fängt keine Pakete auf, da alle Pakete über die VPN Verbindung verschlüsselt sind.

filter: udp and ip.src==192.168.1.8 and ip.dst==136.199.224.0/24

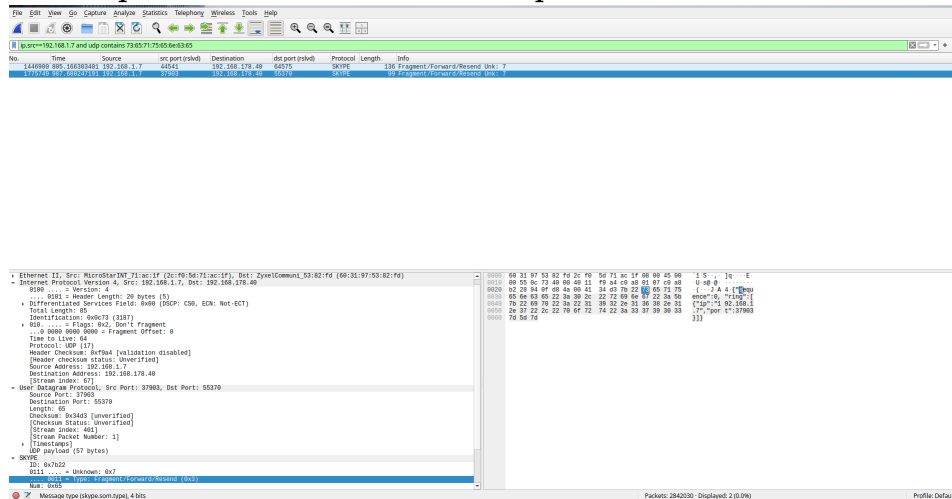


Dieser Filter lässt nur udp Pakete durch, die von der Ursprungsadresse 192.168.1.8 und eine beliebige Zieladresse im Subnet 136.199.224.0 gesendet werden.

Ring: Internet

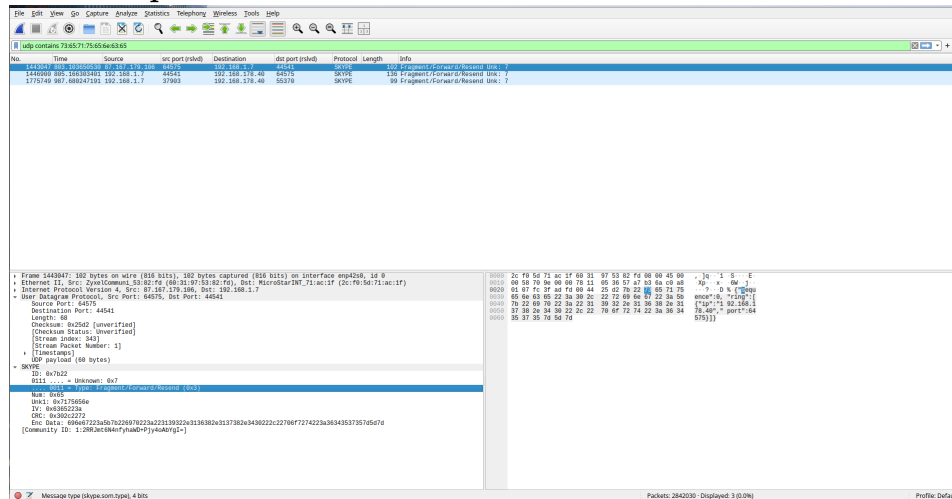
display filter

filter: **ip.src==192.168.1.7 and udp contains 73:65:71:75:65:6e:63:65**



Dieser Anzeigefilter zeigt nur udp Pakete an, die von der Ursprungsadresse 192.168.1.7 gesendet werden und die Bytefolge 73:65:71:75:65:6e:63:65 enthalten.

filter: **udp contains 73:65:71:75:65:6e:63:65**



Dieser Anzeigefilter zeigt nur udp Pakete an, die die Bytefolge 73:65:71:75:65:6e:63:65 enthalten.

No.	Time	Source	src port (ethd)	Destination	dst port (ethd)	Protocol	Length	Info
1	8.900000000	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
2	9.019077377	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
3	9.068875915	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
4	9.140540499	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
5	9.192944999	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
6	9.240649499	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
7	9.289933053	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
8	9.339891815	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
9	9.384024159	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
10	9.432114899	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
11	9.480212169	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
12	9.527324445	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
13	9.572373789	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
14	9.620462057	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
15	9.668567744	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
16	9.716654289	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7
17	9.764741859	192.168.1.5	80411	192.168.1.7	52099	SKYPE	136	Fragment/Forward/Resend: 0x=7

[illegible]

Display Filter

filter: udp.port==52809 && udp contains 72:69:6e:67

The screenshot shows the Wireshark interface with a display filter of `udp.port==52809 && udp contains 72:69:6e:67`. The packet list shows 58991 packets. The selected packet (No. 58991) is a UDP packet from 192.168.1.7 to 192.168.1.5 on port 52809. The packet details show the UDP payload (94 bytes) containing the hex sequence 72:69:6e:67.

Dieser Anzeige Filter filtert alle Pakete, die keine UDP Pakete sind, nicht Port 52809 verwenden und die nicht die Bytefolge 72:69:6e:67 (*Ring*) enthalten.

filter: (ip.dst==192.168.1.7 || ip.dst==192.168.1.5) && udp.length==102

The screenshot shows the Wireshark interface with a display filter of `(ip.dst==192.168.1.7 || ip.dst==192.168.1.5) && udp.length==102`. The packet list shows 58991 packets. The selected packet (No. 58991) is a UDP packet from 192.168.1.7 to 192.168.1.5 on port 52809. The packet details show the UDP payload (94 bytes) containing the hex sequence 72:69:6e:67.

Dieser Anzeigefilter filtert alle Pakete, die weder an die Zieladresse 192.168.1.5 noch an die Zieladresse 192.168.1.7 gehen und nicht 102 byte lang sind.

Probleme:

Beim Versuch den Tokenring über das Internet aufzubauen konnte der 2. Teilnehmer kein *port forwarding* einrichten, weshalb der Ring nicht zustande kam. Zwar konnte ich Daten von ihm empfangen (siehe oben), da ich für den entsprechenden Port eine Weiterleitung eingerichtet hatte, aber er hat die Antwort nicht empfangen können, da sein Router das Paket nicht weitergeleitet hat.

Der Versuch, den Tokenring lokal an der Universität ohne Verwendung des VPN Zuganges aufzubauen, schlug ebenfalls fehl. Es wurden Pakete zwar gesendet, diese kamen jedoch nicht bei dem Empfänger an, da (vermutlich) die entsprechenden ports an den Access Points gesperrt sind. Der Versuch den Socket an einen offenen Port wie 4500 zu binden, schlug ebenfalls fehl, da diese bereits besetzt waren. Da wir keinen Zugriff auf die Access Points haben, haben wir keine weiteren Versuche unternommen.

Der Versuch, einen Tokenring über den VPN der Universität aufzubauen, war sofort erfolgreich. Der Versuch, den Tokenring lokal in meinem Heimnetzwerk aufzubauen, war ebenfalls sofort erfolgreich.